

| 1 lexical analyzer

| algorithm

The code is a lexical analyzer that reads from an input file (`inp.txt`) and identifies keywords, operators, and identifiers, writing results to an output file (`output.txt`).

| Steps

1. Initialize Variables:

- Define arrays for storing keywords and operators.
- Open the input file (`inp.txt`) and output file (`output.txt`).
- Initialize counters for line numbers (`l`) and identifiers (`index`).

2. File Reading:

- Check if the input file is successfully opened; if not, exit with an error message.
- Start reading each character from the input file until reaching the end-of-file (EOF).

3. Token Classification:

- For each character read:
 - Arithmetic Operators:
 - Check if the character is an arithmetic operator (`+` , `-` , `*` , `/` , `%`).
 - If yes, classify it as an *arithmetic operator* and print/write it along with its line number.
 - Alphanumeric Characters:
 - If the character is alphanumeric:
 - Build a potential identifier by reading consecutive alphanumeric characters into a buffer.
 - After reading, check if the buffer contains a keyword.
 - If yes, classify it as a *keyword*.
 - If it's not a keyword, classify it as an *identifier*:
 - New Identifiers:
 - Assign a unique identifier number, write it to the output file, and store it for later references.
 - Existing Identifiers:
 - Retrieve its identifier number from the output file and classify accordingly.
 - Relational Operators:
 - Check for relational operators (`<` , `<=` , `>` , `>=` , `==`).

- Classify them as relational operators and print/write them with their line number.
- Assignment Operator:
 - If the character is `=`, check if it forms `==` (relational equality) or just `=` (assignment).
 - Classify accordingly.

4. Line Counting:

- If a newline (`\n`) character is encountered, increment the line counter (`l`).

5. End of File:

- After processing all characters, close both input and output files.

6. Display Results:

- Print and write the final lexeme classifications (tokens and line numbers) to both the console and the output file.

code

```

1  #include<stdio.h>
2  #include<stdlib.h>
3  #include<string.h>
4  #include<ctype.h>
5
6  int isKeyword(char buffer[]) {
7      char keywords[35][10] = {
8          "auto",
9          "break",
10         "case",
11         "char",
12         "const",
13         "continue",
14         "default",
15         "do",
16         "double",
17         "else",
18         "enum",
19         "extern",
20         "float",
21         "for",
22         "goto",
23         "if",
24         "int",
25         "long",
26         "register",
27         "return",
28         "short",
29         "signed",
30         "sizeof",

```

```

31     "static",
32     "struct",
33     "switch",
34     "typedef",
35     "union",
36     "unsigned",
37     "void",
38     "volatile",
39     "while",
40     "printf",
41     "scanf",
42     "main"
43 };
44
45 int i, flag = 0;
46 for (i = 0; i < 35; i++) {
47     if (strcmp(keywords[i], buffer) == 0) {
48         flag = 1;
49         break;
50     }
51 }
52 return flag;
53 }
54
55 void main() {
56     char ch, ch1, buffer[50], operator[] = "+-*/%";
57     FILE * fp, * fp2;
58     int i, j = 0, l = 1, index = 1;
59     char arr[1000][3];
60     fp = fopen("inp.txt", "r");
61     fp2 = fopen("output.txt", "w+");
62     if (fp == NULL) {
63         printf("Error while opening file\n");
64         exit(0);
65     } else {
66         printf("Lexeme: \t L.no:\t Token:");
67         fprintf(fp2, "Lexeme: \t L.no:\t Token:");
68         while ((ch = fgetc(fp)) != EOF) {
69             for (i = 0; i < 5; ++i) {
70                 if (ch == operator[i]) {
71                     printf("\n%c \t\t %d\t Arithmetic Operator ", ch, l);
72                     break;
73                 }
74             }
75             if (i == 5) {
76                 if (isalnum(ch)) {
77                     if (isdigit(ch) && j == 0) {
78                         j = 0;
79                     } else {
80                         buffer[j++] = ch;

```

```

81     ch1 = fgetc(fp);
82     if (isalnum(ch1))
83         fseek(fp, -1, SEEK_CUR);
84     else {
85         fseek(fp, -1, SEEK_CUR);
86         buffer[j] = '\0';
87         if (isKeyword(buffer) == 1) {
88             if (strcmp(buffer, "printf") == 0) {
89                 while ((ch = fgetc(fp)) != '\n') {
90
91                 }
92             }
93             printf("\n%s \t\t %d \t keyword ", buffer, l);
94             j = 0;
95         } else {
96             if (strcmp(buffer, "main") != 0)
97
98             {
99                 fseek(fp2, SEEK_SET, 0);
100                 char a[50], b[50], c[50], ch2, ch3;
101                 int flag1 = 0;
102                 while (!feof(fp2)) {
103                     fscanf(fp2, "%s\t%s\t%s", a, b, c);
104                     if (strcmp(a, buffer) == 0) {
105                         flag1 = 1;
106                         break;
107                     }
108                 }
109                 if (flag1 == 0) {
110                     ch2 = fgetc(fp);
111                     if (ch2 == '=') {
112                         ch3 = fgetc(fp);
113                         fprintf(fp2, "\n%s \t\t %d\t identifier, %c",
buffer, index, ch3);
114                         fseek(fp, -2, SEEK_CUR);
115                     } else {
116                         fprintf(fp2, "\n%s \t\t %d\t identifier", buffer,
index);
117                     }
118                     printf("\n%s \t\t %d\t Identifier, %d", buffer, l,
index);
119                     index++;
120                 } else {
121                     printf("\n%s \t\t %d\t Identifier, %s", buffer, l,
b);
122                 }
123             }
124             j = 0;
125         }
126     }

```

```

127     }
128 }
129 if (ch == '<') {
130     ch1 = fgetc(fp);
131     if (ch1 == '=') {
132         printf("\n%c%c \t\t %d \t Relop LE", ch, ch1, l);
133     } else {
134         fseek(fp, -1, SEEK_CUR);
135         printf("\n%c \t\t %d \t RelOP LT", ch, l);
136     }
137 } else if (ch == '>') {
138     ch1 = fgetc(fp);
139     if (ch1 == '=') {
140         printf("\n%c%c \t\t %d \t Relop GE", ch, ch1, l);
141     } else {
142         fseek(fp, -1, SEEK_CUR);
143         printf("\n%c \t\t %d \t Relop GT", ch, l);
144     }
145 } else if (ch == '=') {
146     ch1 = fgetc(fp);
147     if (ch1 == '=') {
148         printf("\n%c%c \t\t %d \t Relop EQ", ch, ch1, l);
149     } else {
150         fseek(fp, -1, SEEK_CUR);
151         printf("\n%c \t\t %d \t Assign OP, EQ", ch, l);
152     }
153 }
154 if (ch == '\n') {
155     l++;
156 }
157 }
158 }
159 }
160 printf("\n");
161 }

```

| 2 operator precedence parser

| algorithm

1. Initialize Stack and Input

- Initialize the stack with a \$ symbol.
- Append \$ to the end of the input string.

2. Display Headers for Parsing Process

- Print "STACK", "INPUT", and "ACTION" to indicate the parsing steps.

3. Parsing Loop

- While there is input to process:
 - Shift Operation
 - Push the current input character onto the stack.
 - Move to the next input character.
 - Print the stack and remaining input with action "Shift".
 - Check Precedence
 - If the precedence of the top stack symbol is > (higher than the input):
 - Reduce Operation
 - Check if the top of the stack matches any predefined handle (e.g., E+E , i , etc.).
 - If a handle is found:
 - Replace the matched sequence on the stack with E .
 - Print the stack, remaining input, and the reduction action.

4. Final Check

- If the stack contains only \$E\$ after all reductions, accept the string.
- Else, reject the string.

| code

```
1  #include<stdio.h>
2  #include<string.h>
3  #include<stdlib.h>
4
5  char * input;
6  int i = 0;
7  char lasthandle[6], stack[50], handles[][7] = {
8      ")E(", "E*E", "E+E", "i", "E^E", "E-E", "E/E"
9  };
10 // (E) becomes )E( when pushed to stack
```

```

11
12 int top = 0, l;
13 char prec[9][9] = {
14                                     /*input*/
15     /*stack   +   -   *   /   ^   i   (   )   $ */
16
17     /* + */ '>', '>', '<', '<', '<', '<', '<', '>', '>',
18
19     /* - */ '>', '>', '<', '<', '<', '<', '<', '>', '>',
20
21     /* * */ '>', '>', '>', '>', '<', '<', '<', '>', '>',
22
23     /* / */ '>', '>', '>', '>', '<', '<', '<', '>', '>',
24
25     /* ^ */ '>', '>', '>', '>', '<', '<', '<', '>', '>',
26
27     /* i */ '>', '>', '>', '>', '>', 'e', 'e', '>', '>',
28
29     /* ( */ '<', '<', '<', '<', '<', '<', '<', '>', 'e',
30
31     /* ) */ '>', '>', '>', '>', '>', 'e', 'e', '>', '>',
32
33     /* $ */ '<', '<', '<', '<', '<', '<', '<', '<', '>',
34
35 };
36
37 int getindex(char c) {
38     switch (c) {
39     case '+':
40         return 0;
41     case '-':
42         return 1;
43     case '*':
44         return 2;
45     case '/':
46         return 3;
47     case '^':
48         return 4;
49     case 'i':
50         return 5;
51     case '(':
52         return 6;
53     case ')':
54         return 7;
55     case '$':
56         return 8;
57     }
58 }
59
60 int shift() {

```

```

61     stack[++top] = * (input + i++);
62     stack[top + 1] = '\0';
63 }
64
65 int reduce() {
66     int len, found, t;
67     for (int i = 0; i < 7; i++) { //selecting handles
68         len = strlen(handles[i]);
69         if (stack[top] == handles[i][0] && top + 1 >= len) {
70             found = 1;
71             for (t = 0; t < len; t++) {
72                 if (stack[top - t] != handles[i][t]) {
73                     found = 0;
74                     break;
75                 }
76             }
77             if (found == 1) {
78                 stack[top - t + 1] = 'E';
79                 top = top - t + 1;
80                 strcpy(lasthandle, handles[i]);
81                 stack[top + 1] = '\0';
82                 return 1; //successful reduction
83             }
84         }
85     }
86     return 0;
87 }
88
89 void dispstack() {
90     for (int j = 0; j <= top; j++)
91         printf("%c", stack[j]);
92 }
93
94 void dispinput() {
95     for (int j = i; j < l; j++)
96         printf("%c", *(input + j));
97 }
98
99 void main() {
100     input = (char * ) malloc(50 * sizeof(char));
101     printf("\nEnter the string\n");
102     scanf("%s", input);
103     input = strcat(input, "$");
104     l = strlen(input);
105     strcpy(stack, "$");
106     printf("\nSTACK\tINPUT\tACTION");
107     while (i <= l) {
108         shift();
109         printf("\n");
110         dispstack();

```

```
111     printf("\t");
112     dispinput();
113     printf("\tShift");
114     if (prec[getindex(stack[top])][getindex(input[i])] == '>') {
115         while (reduce()) {
116             printf("\n");
117             dispstack();
118             printf("\t");
119             dispinput();
120             printf("\tReduced: E->%s", lasthandle);
121         }
122     }
123 }
124
125 if (strcmp(stack, "$E$") == 0)
126     printf("\nAccepted;");
127 else
128     printf("\nNot Accepted;");
129 }
```

| 3 recursive descent parser

| algorithm

1. Initialize Variables

- Set $i = 0$ as the index for tracking the current character in the input.
- Set $err = 0$ to track if any parsing error occurs.

2. Start Parsing Expression

- Call the function $E()$ to begin parsing the expression.

3. Define Parsing Rules (Functions)

- The functions E , E_{prime} , T , T_{prime} , and F define the grammar rules for the expression.

| Functions:

- E (Expression)
 - Call $T()$ to parse a term.
 - Call $E_{prime}()$ to handle any $+$ or $-$ operators following the term.
- E_{prime} (Expression Prime)
 - If the current character is $+$ or $-$, increment i to move to the next character.
 - Call $T()$ to parse the next term.
 - Call $E_{prime}()$ recursively to continue parsing additional $+$ or $-$ operators.
- T (Term)
 - Call $F()$ to parse a factor.
 - Call $T_{prime}()$ to handle any $*$ or $/$ operators following the factor.
- T_{prime} (Term Prime)
 - If the current character is $*$ or $/$, increment i .
 - Call $F()$ to parse the next factor.
 - Call $T_{prime}()$ recursively to continue parsing additional $*$ or $/$ operators.
- F (Factor)
 - If the current character is an alphanumeric character, increment i to move to the next character.
 - If the current character is $($, increment i and call $E()$ to parse the sub-expression inside parentheses.
 - After parsing the sub-expression, check for a closing $)$; if absent, set $err = 1$ to indicate an error.

4. Check Result

- If `i` points to the end of the input string (`strlen(input) == i`) and `err == 0` , the input expression is valid.
- Otherwise, the expression is invalid.

5. Output Result

- Print a message indicating whether the expression is accepted or rejected based on the validation result.

Example Execution

1. User inputs an arithmetic expression (e.g., `a+b*c`).
2. The program parses each character according to the grammar rules.
3. If parsing is successful without errors, the expression is accepted; otherwise, it is rejected.

This parser supports basic arithmetic operations (`+` , `-` , `*` , `/`) and parentheses.

code

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  char input[100];
6  int i, err;
7
8  void E();
9  void Eprime();
10 void T();
11 void Tprime();
12 void F();
13
14 void E() {
15     T();
16     Eprime();
17 }
18
19 void Eprime() {
20     if (input[i] == '+') {
21         i++;
22         T();
23         Eprime();
24     } else if (input[i] == '-') {
25         i++;
26         T();
27         Eprime();
28     }
```

```

29  }
30
31  void T() {
32      F();
33      Tprime();
34  }
35
36  void Tprime() {
37      if (input[i] == '*') {
38          i++;
39          F();
40          Tprime();
41      } else if (input[i] == '/') {
42          i++;
43          F();
44          Tprime();
45      }
46  }
47
48  void F() {
49      if (isalnum(input[i])) {
50          i++;
51      } else if (input[i] == '(') {
52          i++;
53          E();
54          if (input[i] == ')') {
55              i++;
56          } else {
57              err = 1;
58          }
59      } else {
60          err = 1;
61      }
62  }
63
64  int main() {
65      i = 0;
66      err = 0;
67      printf("Enter an expression: ");
68      fgets(input, sizeof(input), stdin);
69
70      // Remove newline character if present
71      input[strcspn(input, "\n")] = '\0';
72
73      E();
74      if (strlen(input) == i && err == 0) {
75          printf("%s is accepted\n", input);
76      } else {
77          printf("%s is rejected\n", input);
78      }

```

```
79
80     return 0;
81 }
```

| 4 first and follow

| algorithm

1. Initialize Variables and Matrices

- `firstmat` : Stores First sets of non-terminals.
- `followmat` : Stores Follow sets of non-terminals.
- `production` : Stores grammar productions.
- Other counters and temporary variables.

2. Input Productions

- Read the number of productions, `count` .
- Store each production string in the `production` array.

3. Compute First Sets

- For each production:
 1. Initialize `firstmat` for each non-terminal.
 2. Check if the First set for the current non-terminal is already computed.
 3. If not, call `findfirst` to compute the First set for the current non-terminal.
 4. Store the computed First set in `firstmat` .
- Print the First set for each non-terminal.

4. Compute Follow Sets

- For each production:
 1. Initialize `followmat` for each non-terminal.
 2. Check if the Follow set for the current non-terminal is already computed.
 3. If not, call `follow` to compute the Follow set for the current non-terminal.
 4. Store the computed Follow set in `followmat` .
- Print the Follow set for each non-terminal.

5. Functions to Compute First and Follow Sets

- `findfirst(c, q1, q2)`:
 - Recursively find the First set of non-terminal `c` .
 - Add terminals and ϵ (epsilon) directly to the First set.
 - For non-terminals, recursively call `findfirst` on the next symbol.
- `follow(c)`:
 - Compute Follow set of non-terminal `c` .
 - If `c` is the start symbol, add `$` to its Follow set.
 - Use helper function `followfirst` to handle indirect Follow derivations.
- `followfirst(c, c1, c2)`:
 - Compute First set for symbols following a given symbol in a production.

6. Output Results

- Print `firstmat` and `followmat`, which contain First and Follow sets for all non-terminals.

code

```
1  #include<stdio.h>
2  #include<ctype.h>
3  #include<string.h>
4
5  void followfirst(char, int, int);
6  void follow(char c);
7  void findfirst(char, int, int);
8
9  int count, n = 0, m = 0, k, e, p1 = 0, p2, tmp;
10 char firstmat[10][100], followmat[10][100], production[10][10], f[10],
    first[10], ck;
11
12 int main(int argc, char argv) {
13     int jm = 0;
14     int km = 0;
15     int i, choice;
16     char c, ch;
17     printf("\nEnter the productions: ");
18
19     scanf("%d", & count);
20     for (i = 0; i < count; i++)
21         scanf("%s", production[i]);
22     int kay;
23     char done[count];
24     int ptr = -1;
25     for (k = 0; k < count; k++) {
26         for (kay = 0; kay < 100; kay++)
27             firstmat[k][kay] = '!';
28     }
29     int p1 = 0, p2, tmp;
30     for (k = 0; k < count; k++) {
31         c = production[k][0];
32         p2 = 0;
33         tmp = 0;
34         for (kay = 0; kay <= ptr; kay++)
35             if (c == done[kay])
36                 tmp = 1;
37         if (tmp == 1)
38             continue;
39         findfirst(c, 0, 0);
40         ptr += 1;
41         done[ptr] = c;
42         printf("\nFirst(%c) = { ", c);
```

```

43     firstmat[p1][p2++] = c;
44     for (i = 0 + jm; i < n; i++) {
45         int lark = 0, chk = 0;
46         for (lark = 0; lark < p2; lark++) {
47             if (first[i] == firstmat[p1][lark]) {
48                 chk = 1;
49                 break;
50             }
51         }
52         if (chk == 0) {
53             printf("%c, ", first[i]);
54             firstmat[p1][p2++] = first[i];
55         }
56     }
57     printf("}\n");
58     jm = n;
59     p1++;
60 }
61 printf("\n\n\n");
62 char donee[count];
63 ptr = -1;
64 for (k = 0; k < count; k++) {
65     for (kay = 0; kay < 100; kay++)
66         followmat[k][kay] = '!';
67 }
68 p1 = 0; //in p
69 int kk, u, flag, land = 0;
70 char s[10];
71 u = 0;
72 flag = 0;
73 for (e = 0; e < count; e++) {
74     kk = 0;
75     ck = production[e][0];
76     for (i = 0; i <= u; i++) {
77         if (s[i] == ck) {
78             kk++;
79         }
80     }
81     if (kk == 0) {
82
83         s[u] = ck;
84         u++;
85         p2 = 0;
86         tmp = 0;
87         for (kay = 0; kay <= ptr; kay++)
88             if (ck == donee[kay])
89                 tmp = 1;
90
91         if (tmp == 1)
92             continue;

```

```

93     land += 1;
94     follow(ck);
95
96     ptr += 1;
97     printf("Follow(%c) = { ", ck);
98     followmat[p1][p2++] = ck;
99     for (i = 0 + km; i < m; i++) {
100         int lark = 0, chk = 0;
101         for (lark = 0; lark < p2; lark++) {
102             if (f[i] == followmat[p1][lark]) {
103                 chk = 1;
104                 break;
105             }
106         }
107         if (chk == 0) {
108             printf("%c, ", f[i]);
109             followmat[p1][p2++] = f[i];
110         }
111     }
112     printf("}\n\n");
113     km = m;
114     p1++;
115 }
116 }
117 }
118
119 void follow(char c) {
120     int i, j;
121     if (production[0][0] == c)
122         f[m++] = '$';
123
124     for (i = 0; i < 10; i++) {
125         for (j = 2; j < 10; j++) {
126             if (production[i][j] == c) {
127                 if (production[i][j + 1] != '\0')
128                     followfirst(production[i][j + 1], i, (j + 2));
129
130                 if (production[i][j + 1] == '\0' && c != production[i][0])
131                     follow(production[i][0]);
132             }
133         }
134     }
135 }
136 }
137
138 void findfirst(char c, int q1, int q2) {
139     int j;
140     if (!(isupper(c)))
141         first[n++] = c;
142     for (j = 0; j < count; j++) {

```

```

143     if (production[j][0] == production[j][2])
144         continue;
145
146     if (production[j][0] == c) {
147         if (production[j][2] == '#') {
148             if (production[q1][q2] == '\0')
149                 first[n++] = '#';
150             else if (production[q1][q2] != '\0' && (q1 != 0 || q2 != 0))
151                 findfirst(production[q1][q2], q1, (q2 + 1));
152             else
153                 first[n++] = '#';
154         } else if (!isupper(production[j][2]))
155             first[n++] = production[j][2];
156         else
157             findfirst(production[j][2], j, 3);
158     }
159 }
160 }
161 void followfirst(char c, int c1, int c2) {
162     int k;
163     if (!(isupper(c)))
164         f[m++] = c;
165     else {
166         int i = 0, j = 1;
167         for (i = 0; i < count; i++) {
168             if (firstmat[i][0] == c)
169                 break;
170         }
171         while (firstmat[i][j] != '!') {
172             if (firstmat[i][j] != '#')
173                 f[m++] = firstmat[i][j];
174             else {
175                 if (production[c1][c2] == '\0')
176                     follow(production[c1][0]);
177                 else
178                     followfirst(production[c1][c2], c1, c2 + 1);
179             }
180             j++;
181         }
182     }
183 }
184

```

| 5 intermediate code generation

| algorithm

| Overview

This program converts an infix expression to a postfix expression and then generates three types of intermediate code representations:

1. Three Address Code
2. Quadruple Representation
3. Triple Representation

| Variables and Structures

- `s` : Array to store the input infix expression.
- `post` : Array to store the postfix expression.
- `stack` : Stack used to hold operators during the conversion process.
- `queue` : Queue for handling operands during the generation of three-address code.
- `top` , `j` , `front` : Variables for indexing and managing stack/queue positions.

| Steps

| Step 1: Define Operator Precedence Function

1. `precedence(char c)` : Returns an integer that represents the precedence of operators `+` , `-` , `*` , `/` , `^` .

| Step 2: Convert Infix to Postfix (Function `postfix`)

1. Read each character in `s` :
 - If character is `(` : Push it onto the `stack` .
 - If character is an operand (variable): Append it to `post` .
 - If character is an operator:
 - Pop operators from `stack` to `post` until a lower-precedence operator is found.
 - Push the current operator onto the `stack` .
 - If character is `)` : Pop operators from `stack` to `post` until `(` is found; then discard `(` .
2. After scanning `s` , pop any remaining operators in `stack` to `post` .

| Step 3: Generate Intermediate Code in Three Formats

1. Open input and output files for reading and writing.
2. For each infix expression in `input.txt` :
 - Convert it to postfix using `postfix()` .
 - Print both infix and postfix forms to console.
3. For each character in `post` :
 - If character is an operator:
 - Pop two operands (`a` and `b`) from `queue` .
 - Generate three-address, quadruple, and triple representations based on operand type (temporary variable or constant).
 - Push the temporary result back onto `queue` .
 - If character is an operand:
 - Push it onto `queue` .
4. Increment the temporary variable (`ind`) as needed to ensure unique identifiers for results in intermediate code.

| Step 4: Close Files

1. After processing all expressions, close the output files.

| Example Execution

1. Input: `(a+b)*c`
2. Output:
 - Infix: `(a+b)*c`
 - Postfix: `ab+c*`
 - Three Address Code: Generates instructions like `t1 = a + b` then `t2 = t1 * c`
 - Quadruple and Triple Representations: Output appropriate entries based on operator and operands used.

| code

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <ctype.h>
5
6  char s[100], post[100], stack[100], queue[100];
7  int top = -1, j = 0, front = -1;
8
9  int precedence(char c) {
10     if (c == '+' || c == '-') {
```

```

11     return 1;
12 } else if (c == '*' || c == '/') {
13     return 2;
14 } else if (c == '^') {
15     return 3;
16 } else {
17     return 0;
18 }
19 }
20
21 void postfix() {
22     int n = strlen(s), i = 0;
23     while (s[i] != '\0') {
24         if (s[i] == '(') {
25             top++;
26             stack[top] = s[i];
27         } else if (isalpha(s[i])) {
28             post[j++] = s[i];
29         } else if (precedence(s[i])) {
30             while (precedence(stack[top]) >= precedence(s[i])) {
31                 post[j++] = stack[top--];
32             }
33             top++;
34             stack[top] = s[i];
35         } else if (s[i] == ')') {
36             while (stack[top] != '(') {
37                 post[j++] = stack[top--];
38             }
39             top--;
40         }
41         i++;
42     }
43     while (top != -1) {
44         post[j++] = stack[top--];
45     }
46     post[j] = '\0';
47 }
48
49 void main() {
50     int i = 0;
51     char ind = '1';
52
53     FILE * fp, * fp1, * fp2, * fp3;
54     fp = fopen("3addr.txt", "w");
55     fp1 = fopen("quadruple.txt", "w");
56     fp2 = fopen("triple.txt", "w");
57     fprintf(fp, "Three Address\n");
58     fprintf(fp1, "Quadruple\nOP\t01\t02\tRES\n");
59     fprintf(fp2, "Triple\nIN\t0P\t01\t02\n");
60     fp3 = fopen("input.txt", "r");

```

```

61     printf("Input the string: ");
62
63     postfix();
64     while ((fscanf(fp3, "%s", s)) != EOF) {
65         postfix();
66         printf("Infix : %s \nPostfix : %s\n", s, post);
67         while (post[i] != '\0') {
68             if (precedence(post[i])) {
69                 char a = queue[front--];
70                 char b = queue[front--];
71                 if (isdigit(a) && isdigit(b)) {
72                     fprintf(fp, "t%c = t%c %c t%c\n", ind, b, post[i], a);
73                     fprintf(fp1, "%c\tt%c\tt%c\tt%c\n", post[i], b, a, ind);
74                 } else if (isdigit(b)) {
75                     fprintf(fp, "t%c = t%c %c %c\n", ind, b, post[i], a);
76                     fprintf(fp1, "%c\tt%c\tt%c\tt%c\n", post[i], b, a, ind);
77                 } else if (isdigit(a)) {
78                     fprintf(fp, "t%c = %c %c t%c\n", ind, b, post[i], a);
79                     fprintf(fp1, "%c\tt%c\tt%c\tt%c\n", post[i], b, a, ind);
80                 } else {
81                     fprintf(fp, "t%c = %c %c %c\n", ind, b, post[i], a);
82                     fprintf(fp1, "%c\tt%c\tt%c\tt%c\n", post[i], b, a, ind);
83                 }
84                 fprintf(fp2, "%c\tt%c\tt%c\tt%c\n", ind, post[i], b, a);
85                 front++;
86                 queue[front] = ind;
87                 ind++;
88             } else {
89                 front++;
90                 queue[front] = post[i];
91             }
92             i++;
93         }
94     }
95 }

```

| input.txt

```

1  a+b*c^d+e
2  a*b+c*d

```

| 3addr.txt

```

1  Three Address
2  t1 = c ^ d
3  t2 = b * t1
4  t3 = a + t2
5  t4 = t3 + e

```

```
6   t5 = a * b
7   t6 = c * d
8   t7 = t5 + t6
```

| quadruple.txt

```
1   Quadruple
2   OP      01      02      RES
3   ^       c       d       t1
4   *       b       t1      t2
5   +       a       t2      t3
6   +       t3      e       t4
7   *       a       b       t5
8   *       c       d       t6
9   +       t5      t6      t7
```

| triple.txt

```
1   Triple
2   IN      OP      01      02
3   1       ^       c       d
4   2       *       b       1
5   3       +       a       2
6   4       +       3       e
7   5       *       a       b
8   6       *       c       d
9   7       +       5       6
```

| 6 constant propagation

| algorithm

| Overview

This program reads C code from a file, identifies constants, evaluates simple arithmetic expressions, and performs constant propagation. It rewrites the code with constant values wherever applicable.

| Data Structures

- Variable Structure: Contains:
 - `name` : Name of the variable.
 - `value` : Constant value of the variable.
 - `isConstant` : Flag indicating if the variable is a constant (1 for true, 0 for false).
- Global Variables:
 - `variables[]` : Array to store variables.
 - `variableCount` : Number of variables currently stored.

| Steps

| Step 1: Define Helper Functions

1. `findVariable(char *name)`:
 - Search for a variable by name.
 - Return its index if found; otherwise, return -1.
2. `addConstant(char *name, float value)`:
 - Check if the variable already exists using `findVariable`.
 - If not, add it to the `variables` array with `isConstant` set to 1.
 - If it exists, update its value and set `isConstant` to 1.
3. `evaluateExpression(float left, char operator, float right)`:
 - Perform the specified arithmetic operation on `left` and `right`.
 - Handle division by zero by returning 0 if the divisor is 0.
4. `parseAssignment(char line, char varName, float *result)`:
 - Use `sscanf` to parse a line for an assignment statement in the format `float varName = left operator right;`.
 - If successful, evaluate the expression and store the result in `*result`.
5. `propagateConstants(char *line)`:

- For each constant variable in `variables` , replace its occurrences in the line with its constant value.
- Adjust the length of the line if the constant value is shorter than the variable name.

6. `parseLine(char *line):`

- Check if the line defines a constant expression using `parseAssignment` .
- If it does, add the constant using `addConstant` and rewrite the line with the constant value.
- If not, call `propagateConstants` to replace constants in the line.

Step 2: Main Function

1. Open the file `cd6result.txt` for reading.
2. Check for file open errors; print error message and exit if needed.
3. Print Original Code: Read and display each line from the file.
4. Reset File Pointer: Use `rewind` to reset the file pointer to the beginning of the file.
5. Perform Constant Propagation:
 - Read each line again and call `parseLine` .
 - Print the modified line with constant values.
6. Close the file.

code

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4  #include <ctype.h>
5
6  #define MAX_VARIABLES 100
7  #define MAX_LINE_LENGTH 256
8
9  // Structure to store variables and their constant values
10 typedef struct {
11     char name[20];
12     float value;
13     int isConstant;
14 }
15 Variable;
16
17 Variable variables[MAX_VARIABLES];
18 int variableCount = 0;
19
20 // Function to find variable index
21 int findVariable(char * name) {
22     for (int i = 0; i < variableCount; i++) {

```

```

23     if (strcmp(variables[i].name, name) == 0) {
24         return i;
25     }
26 }
27 return -1;
28 }
29
30 // Function to add or update a variable as a constant
31 void addConstant(char * name, float value) {
32     int index = findVariable(name);
33     if (index == -1) {
34         strcpy(variables[variableCount].name, name);
35         variables[variableCount].value = value;
36         variables[variableCount].isConstant = 1;
37         variableCount++;
38     } else {
39         variables[index].value = value;
40         variables[index].isConstant = 1;
41     }
42 }
43
44 // Function to evaluate basic expressions (supports +, -, *, /)
45 float evaluateExpression(float left, char operator, float right) {
46     switch (operator) {
47         case '+':
48             return left + right;
49         case '-':
50             return left - right;
51         case '*':
52             return left * right;
53         case '/':
54             return right != 0 ? left / right : 0; // Check for divide-by-zero
55         default:
56             return 0;
57     }
58 }
59
60 // Function to parse and evaluate assignments like 'pi = 20 / 5'
61 int parseAssignment(char * line, char * varName, float * result) {
62     int left, right;
63     char operator;
64
65     if (sscanf(line, " float %s = %d %c %d ;", varName, & left, &
operator, & right) == 4) {
66         * result = evaluateExpression((float) left, operator, (float)
right);
67         return 1;
68     }
69     return 0;
70 }

```

```

71
72 // Function to perform constant propagation
73 void propagateConstants(char * line) {
74     for (int i = 0; i < variableCount; i++) {
75         if (variables[i].isConstant) {
76             char varName[20];
77             sprintf(varName, "%s", variables[i].name);
78
79             char constValue[20];
80             sprintf(constValue, "%.2f", variables[i].value);
81
82             char * pos = strstr(line, varName);
83             while (pos != NULL) {
84                 strncpy(pos, constValue, strlen(constValue));
85                 if (strlen(varName) > strlen(constValue)) {
86                     memmove(pos + strlen(constValue), pos + strlen(varName),
87                             strlen(pos + strlen(varName)) + 1);
88                     pos = strstr(line, varName); // Check for other occurrences
89                 }
90             }
91         }
92     }
93
94 // Function to parse each line for constants or propagation
95 void parseLine(char * line) {
96     char varName[20];
97     float result;
98
99     // Check if line defines a constant expression
100    if (parseAssignment(line, varName, & result)) {
101        addConstant(varName, result);
102
103        // Rewrite line with constant value
104        sprintf(line, "float %s = %.2f;\n", varName, result);
105    } else {
106        propagateConstants(line);
107    }
108 }
109
110 int main() {
111     FILE * file = fopen("cd6result.txt", "r");
112     if (file == NULL) {
113         perror("Error opening file");
114         return 1;
115     }
116
117     char line[MAX_LINE_LENGTH];
118
119     printf("Original Code:\n");

```

```

120     while (fgets(line, sizeof(line), file)) {
121         printf("%s", line);
122     }
123     rewind(file); // Reset file pointer to beginning
124
125     printf("\nAfter Constant Propagation:\n");
126     while (fgets(line, sizeof(line), file)) {
127         parseLine(line);
128         printf("%s", line);
129     }
130
131     fclose(file);
132     return 0;
133 }

```

| input.txt

```

1  float pi=20/5;
2  void main()
3  {
4      float c;
5      c=2*pi;
6  }
7

```

| cons.txt

```

1  8.000000

```

| result.txt

```

1  float pi=4.000000;
2  void main()
3  {
4      float c;
5      c=8.000000;
6  }
7

```

| 7 common subexpression elimination

| algorithm

| Input:

- A filename containing three-address code expressions.

| Data Structure:

- Define a structure `Expression` to store:
 - `lhs` : Left-hand side variable (e.g., `t1`, `t2`)
 - `op` : Operator (e.g., `+`, `*`, `^`, `-`, `/`)
 - `rhs1` : First operand (can be a variable or another temporary)
 - `rhs2` : Second operand (can be a variable or another temporary)

| Steps:

1. Read Input:

- Prompt the user for the filename.
- Open the file for reading.
- Initialize an array `exp` to store up to `MAX_EXPRESSIONS`.
- Read expressions from the file into the array:
 - For each line in the file, extract `lhs`, `rhs1`, `op`, and `rhs2` using `sscanf`.
 - Increment the count of expressions.

2. Check for Expressions:

- If no expressions were read, print a message and exit.

3. Eliminate Common Subexpressions:

- For each expression at index `i` in the array:
 1. Call `findCommonSubexpression(exp, i, exp[i])` to check for common subexpressions.
 - If found, store the index of the common expression.
 2. If a common expression is found:
 - Print details of the common subexpression.
 - If the left-hand side (`lhs`) of the current expression is not `"t6"`:
 - Replace occurrences of the eliminated expression using `replaceOccurrences`.
 - Remove the current expression from the array by shifting elements.

- Decrement the count of expressions and recheck the current index.

4. Output Optimized Expressions:

- Print the remaining expressions after optimization.

Output:

- The optimized list of expressions after eliminating common subexpressions.

code

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4  #define MAX_EXPRESSIONS 100
5
6  // Structure to represent an expression in three-address code
7  struct Expression {
8      char lhs[10]; // Left-hand side variable (e.g., t1, t2)
9      char op[3]; // Operator (e.g., +, *, ^, -, /)
10     char rhs1[10]; // First operand (can be a variable or another
        temporary)
11     char rhs2[10]; // Second operand (can be a variable or another
        temporary)
12 };
13
14 int findCommonSubexpression(struct Expression exp[], int n, struct
    Expression current) {
15     for (int i = 0; i < n; i++) {
16         if (strcmp(exp[i].op, current.op) == 0 &&
17             strcmp(exp[i].rhs1, current.rhs1) == 0 &&
18             strcmp(exp[i].rhs2, current.rhs2) == 0) {
19             return i; // Return the index of the common expression
20         }
21     }
22     return -1; // No common subexpression found
23 }
24
25 void replaceOccurrences(struct Expression exp[], int n,
    const char * oldVar,
26     const char * newVar) {
27     for (int i = 0; i < n; i++) {
28         if (strcmp(exp[i].rhs1, oldVar) == 0) {
29             strcpy(exp[i].rhs1, newVar);
30         }
31         if (strcmp(exp[i].rhs2, oldVar) == 0) {
32             strcpy(exp[i].rhs2, newVar);
33         }
34     }

```

```

35     }
36 }
37
38 void eliminateCommonSubexpressions(struct Expression exp[], int * n) {
39     for (int i = 0; i < * n; i++) {
40         int index = findCommonSubexpression(exp, i, exp[i]);
41         if (index != -1) {
42             // Print what the common subexpression is
43             printf("Common subexpression found: %s = %s %s %s and %s = %s %s\n",
44                 exp[index].lhs, exp[index].rhs1, exp[index].op,
45                 exp[i].lhs, exp[i].rhs1, exp[i].op, exp[i].rhs2);
46             printf("Eliminating %s = %s %s %s and using %s instead\n",
47                 exp[i].lhs, exp[i].rhs1, exp[i].op, exp[i].rhs2,
48                 exp[index].lhs);
49             // Check if the eliminated expression is t6 before replacing
50             // occurrences
51             if (strcmp(exp[i].lhs, "t6") != 0) {
52                 // Replace occurrences of the eliminated expression
53                 replaceOccurrences(exp, * n, exp[i].lhs, exp[index].lhs);
54             }
55             // Remove the duplicate expression only if it is not t6
56             if (strcmp(exp[i].lhs, "t6") != 0) {
57                 for (int j = i; j < * n - 1; j++) {
58                     exp[j] = exp[j + 1];
59                 }
60                 (* n) --; // Reduce the total number of expressions
61                 i--; // Recheck the current index
62             }
63         }
64     }
65
66     // Output the optimized expression list
67     printf("\nOptimized expressions:\n");
68     for (int i = 0; i < * n; i++) {
69         printf("%s = %s %s %s\n", exp[i].lhs, exp[i].rhs1, exp[i].op,
70             exp[i].rhs2);
71     }
72 }
73
74 int main() {
75     FILE * file;
76     char filename[100];
77     struct Expression exp[MAX_EXPRESSIONS];
78     int n = 0;
79     char line[100];

```

```

80     printf("Enter the filename: ");
81     scanf("%s", filename);
82
83     file = fopen(filename, "r");
84     if (file == NULL) {
85         printf("Could not open file %s\n", filename);
86         return 1;
87     }
88
89     // Read expressions from the file
90     while (fgets(line, sizeof(line), file)) {
91         if (sscanf(line, " %s = %s %s %s", exp[n].lhs, exp[n].rhs1,
exp[n].op, exp[n].rhs2) == 4) {
92             n++; // Increment the count of expressions
93             if (n >= MAX_EXPRESSIONS) {
94                 printf("Warning: Maximum number of expressions reached.\n");
95                 break;
96             }
97         }
98     }
99
100    fclose(file);
101
102    if (n == 0) {
103        printf("No expressions found in the file.\n");
104        return 1;
105    }
106
107    eliminateCommonSubexpressions(exp, & n);
108
109    return 0;
110 }

```

| input.txt

```

1  t1 = c * d
2  t2 = b + t1
3  t3 = c * d
4  t4 = a - t3
5  t5 = d = 10
6  t6 = c * d
7  t7 = e / t6

```

| 8 code generator

| algorithm

1. Initialization

- Declare string variables `opr` , `fir` , `sec` , `res` .
- Initialize integer variables `i` (to 0) and `k` (to 0).
- Declare file pointers `fp1` and `fp2` .

2. Open Files

- Open `input.txt` for reading (`fp1`).
- Open `result.txt` for writing (`fp2`).
- Print "file opened".

3. Read Input

- While not end-of-file (EOF) for `fp1` :
 - Read a character `op` and three strings `fir` , `sec` , `res` from the file.

4. Determine Operation

- Use a switch statement on `op` :
 - If `op` is '+', set `opr` to "ADD".
 - If `op` is '-', set `opr` to "SUB".
 - If `op` is '*', set `opr` to "MUL".
 - If `op` is '/', set `opr` to "DIV".

5. Process First Operand (`fir`)

- Check the length of `fir` :
 - If it has 2 characters, check the second character and write the corresponding `MOV` instruction to `fp2` .
 - Otherwise, write `MOV AX,[fir]` to `fp2` .

6. Process Second Operand (`sec`)

- Check the length of `sec` :
 - If it has 2 characters, check the second character and write the corresponding `MOV` instruction to `fp2` .
 - Otherwise, write `MOV BX,[sec]` to `fp2` .

7. Write Operation

- If `i` is 0, write the operation using `AX` and `BX` (e.g., "ADD AX, BX").
- If `i` is 1, write the operation using only `BX` .

8. Process Result (`res`)

- Check the length of `res` :
 - If it has 2 characters, check the second character and write the corresponding `MOV` instruction to `fp2` .

9. Increment Counter

- Increment `k`.

10. Close Files

- Close `fp1` and `fp2` (not shown in the code but should be done to prevent resource leaks).

code

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4
5  void main() {
6      char opr[10], fir[10], sec[10], res[10];
7      char op;
8      int i = 0, k=0;
9      FILE *fp1 = fopen("input.txt","r");
10     FILE *fp2 = fopen("result.txt","w");
11
12     while (!feof(fp1)){
13         fscanf(fp1, "%c%s%s%s\n",&op, fir, sec, res);
14         printf("%c %s %s %s\n",op, fir, sec, res);
15
16         switch (op){
17             case '+':
18                 strcpy(opr, "ADD ");
19                 i = 0;
20                 break;
21             case '-':
22                 strcpy(opr, "SUB");
23                 i = 0;
24                 break;
25             case '*':
26                 strcpy(opr, "MUL");
27                 i = 1;
28                 break;
29             case '/' :
30                 strcpy(opr, "DIV");
31                 i = 1;
32                 break;
33         }
34         if (strlen(fir) == 2) {
35             if (fir[1] == '1')
36                 fprintf(fp2, "MOV AX,CH\n");
37             else if (fir[1] == '2')
38                 fprintf(fp2, "MOV BX,CL\n");
39             else if (fir[1] == '3')
40                 fprintf(fp2, "MOV CX,DH\n");
```

```

41         else if (fir[1] == '4')
42             fprintf(fp2, "MOV DX,DL\n");
43     } else
44         fprintf(fp2, "MOV AX,[%s]\n", fir);
45     if (strlen(sec) == 2) {
46         if (sec[1] == '1')
47             fprintf(fp2, "MOV BX,CH\n");
48         else if (sec[1] == '2')
49             fprintf(fp2, "MOV BX,CL\n");
50         else if (sec[1] == '3')
51             fprintf(fp2, "MOV BX,DH\n");
52         else if (sec[1] == '4')
53             fprintf(fp2, "MOV BX,DL\n");
54     } else
55         fprintf(fp2, "MOV BX,[%s]\n", sec);
56     if (i==0)
57         fprintf(fp2, "%sAX,BX\n", opr);
58     else
59         fprintf(fp2, "%s BX\n", opr);
60     if(strlen(res) == 2) {
61         if (res[1] == '1')
62             fprintf(fp2, "MOV CH,AX\n");
63         else if (res[1] == '2')
64             fprintf(fp2, "MOV CL,AX\n");
65         else if (res[1] == '3')
66             fprintf(fp2, "MOV DH,AX\n");
67         else if (res[1] == '4')
68             fprintf(fp2, "MOV DL,AX\n");
69     }
70     k++;
71 }
72 }

```

input.txt

```

1  + a b t1
2  + c d t2
3  / t1 t2 t3

```

result.txt

```

1  MOV AX,[a]
2  MOV BX,[b]
3  ADD AX,BX
4  MOV CH,AX
5  MOV AX,[c]
6  MOV BX,[d]
7  ADD AX,BX

```

```
8  MOV CL,AX
9  MOV AX,CH
10 MOV BX,CL
11 DIV BX
```

| 9 Epsilon Closure

| algorithm

This algorithm manages an NFA (Nondeterministic Finite Automaton) with both regular and epsilon transitions. It calculates the epsilon closure for each state, which includes all reachable states via epsilon transitions.

| Steps

| 1. Initialize the NFA Structure

- Define `transition` and `epsilon` tables for regular and epsilon transitions.
- Define the number of states and input symbols (`numStates` and `numSymbols`).

| 2. Input Transitions

- For each state:
 - For each symbol:
 - Prompt the user for transitions to other states.
 - If the symbol is the last one (`#`), treat it as an epsilon transition. Otherwise, record it as a regular transition.

| 3. Compute Epsilon Closure

- For each state:
 - Initialize an empty `closure` array and a `closureSize` counter.
 - Add the current state to the `closure` .
 - Recursively add states reachable by epsilon transitions to `closure` .
 - Print the epsilon closure for each state.

| Function Breakdown

1. **addTransition**: Adds a regular transition between states for a given symbol.
2. **addEpsilonTransition**: Adds an epsilon transition between states.
3. **epsilonClosure**: Computes the epsilon closure for a state by:
 - Adding the state to `closure` .
 - Recursively including any state reachable via epsilon transitions.

| Example Execution

- The program prompts for the number of states and symbols.

- It asks for transitions, where `-1` marks the end of input for each symbol.
- Finally, it outputs the epsilon closure for each state.

code

```

1  #include <stdio.h>
2
3  #define MAX_STATES 100
4  #define MAX_SYMBOLS 100
5  typedef struct {
6      int transition[MAX_STATES][MAX_SYMBOLS][MAX_STATES];
7      int epsilon[MAX_STATES][MAX_STATES];
8      int numStates;
9      int numSymbols;
10 } NFA;
11
12 void addTransition(NFA* nfa, int from, int to, int symbol) {
13     nfa->transition[from][symbol][to] = 1;
14 }
15
16 void addEpsilonTransition(NFA* nfa, int from, int to) {
17     nfa->epsilon[from][to] = 1;
18 }
19
20 void epsilonClosure(NFA* nfa, int state, int* closure, int*
closureSize) {
21     closure[*closureSize] = state;
22
23     (*closureSize)++;
24
25     // Check for epsilon transitions
26     for (int i = 0; i < nfa->numStates; i++) {
27         if (nfa->epsilon[state][i]) {
28             // Check if state is already in closure
29             int alreadyInClosure = 0;
30
31             for (int j = 0; j < *closureSize; j++) {
32                 if (closure[j] == i) {
33                     alreadyInClosure = 1;
34                     break;
35                 }
36             }
37             if (!alreadyInClosure) {
38                 epsilonClosure(nfa, i, closure, closureSize);
39             }
40         }
41     }
42 }
43

```

```

44 int main() {
45     NFA nfa;
46     int epsilonClosureResult[MAX_STATES];
47
48     printf("Enter the number of states: ");
49     scanf("%d", &nfa.numStates);
50
51     printf("Enter the number of input symbols: ");
52     scanf("%d", &nfa.numSymbols);
53
54     for (int i = 0; i < nfa.numStates; i++) {
55         for (int j = 0; j < nfa.numSymbols; j++) {
56             for (int k = 0; k < nfa.numStates; k++) {
57                 nfa.transition[i][j][k] = 0; // No transitions
58             }
59         }
60
61         for (int k = 0; k < nfa.numStates; k++) {
62             nfa.epsilon[i][k] = 0; // No epsilon transitions
63         }
64     }
65
66     for (int i = 0; i < nfa.numStates; i++) {
67         printf("\nEnter the transitions for state %d\n", i);
68
69         for (int j = 0; j < nfa.numSymbols; j++) {
70             printf("\t for symbol %c: ", j == nfa.numSymbols - 1 ? 35 : j +
97);
71
72             int toState;
73             while (1) {
74                 scanf("%d", &toState);
75                 if (toState == -1) break;
76                 if (j == nfa.numSymbols - 1)
77                     addEpsilonTransition(&nfa, i, toState);
78                 else
79                     addTransition(&nfa, i, toState, j);
80             }
81         }
82     }
83
84     for (int i = 0; i < nfa.numStates; i++) {
85         int closure[MAX_STATES];
86         int closureSize = 0;
87
88         epsilonClosure(&nfa, i, closure, &closureSize);
89
90         printf("Epsilon closure of state %d: { ", i);
91         for (int j = 0; j < closureSize; j++) {
92             printf("%d ", closure[j]);

```

```
93     }
94     printf("}\n");
95 }
96
97 return 0;
98 }
```

| 10 E-NFA to DFA

| Algorithm to Convert NFA with Epsilon Transitions to DFA

1. Initialize Variables:

- Define states, terminals, and the transition table.
- Identify and store states that have epsilon transitions.

2. Input Collection:

- Get the number of states and terminals from the user.
- Read states, terminals, and fill the transition table.
- Store final state information (whether each state is accepting or not).

3. Calculate Epsilon Closure:

- For each state with an epsilon transition:
 - Recursively find all states reachable through epsilon transitions (epsilon closure).
 - Update the transition table to account for epsilon closures.

4. Create DFA State Transitions:

- Initialize the DFA starting state as the epsilon closure of the NFA start state.
- For each new DFA state:
 - For each terminal, compute the union of transitions for all NFA states in the DFA state.
 - Check if this new state combination already exists. If not, add it to the list of DFA states.
- Mark DFA states as accepting if any contained NFA state is accepting.

5. Output the DFA:

- Print the DFA transition table with new states represented by letters (e.g., A, B, C).
- Indicate final (accepting) states in the output.

| Explanation of Key Functions

- **Epsilon Closure:** Computes all reachable states for a given state using only epsilon transitions.
- **Sorting for Consistency:** Sorts the characters in each state combination to ensure uniqueness when comparing states.
- **DFA State Generation:** Adds each unique DFA state and its transitions to the final DFA table, marking states as accepting if they contain any accepting NFA states.

| code

```

1  #include<ctype.h>
2  #include<stdio.h>
3  #include<stdlib.h>
4  #include<string.h>
5  #include<stdbool.h>
6
7  char states[100], terms[100], stack[50];
8  int isfinal[100], resfinal[100];
9  int ind = 0, stacklen = 0;
10 char grams[20];
11 char rhs[20][50];
12 int count = 0, rescount = 0, resind = 0;
13 int sn = 0, tn = 0;
14
15 char resStates[100][20], restable[100][20][10];
16
17 char table[50][50][10];
18 char hasEps[100];
19 int eind = 0;
20
21 char eclsr[50];
22 int clsind = 0;
23
24 char sstr[30];
25 int sortString() {
26     int l = strlen(sstr);
27     for (int i = 0; i < l; i++) {
28         for (int j = 0; j < l - 1 - i; j++) {
29             if (sstr[j] > sstr[j + 1]) {
30                 char tem = sstr[j];
31                 sstr[j] = sstr[j + 1];
32                 sstr[j + 1] = tem;
33             }
34         }
35     }
36 }
37
38 int eclfnl = 0;
39
40 int eClosure(char c) {
41     for (int i = 0; i < sn; i++) {
42         if (states[i] == c) {
43             eclfnl = eclfnl | isfinal[i];
44             if (strcmp(table[i][tn], "-") != 0) {
45                 strcat(eclsr, table[i][tn]);
46                 clsind += strlen(table[i][tn]);
47                 for (int j = 0; j < strlen(table[i][tn]); j++) {
48                     eClosure(table[i][tn][j]);
49                 }
50             }

```

```

51         break;
52     }
53 }
54 return 0;
55 }
56
57 int main() {
58     printf("Number of states\t");
59     scanf("%d", & sn);
60     printf("Number of terminals\t");
61     scanf("%d", & tn);
62     getchar();
63     printf("Enter States\t");
64     for (int i = 0; i < sn; i++) {
65         states[i] = getchar();
66         getchar();
67     }
68     printf("Enter terminals\t");
69     for (int i = 0; i < tn; i++) {
70         terms[i] = getchar();
71         getchar();
72     }
73     printf("Transition table\n");
74     for (int i = 0; i < tn; i++) {
75         printf("\t%c", terms[i]);
76     }
77     printf("\tEps\n\tFinal?\n");
78     for (int i = 0; i < sn; i++) {
79         printf("%c\t", states[i]);
80         for (int j = 0; j < tn + 2; j++) {
81             scanf("%s", table[i][j]);
82             if (j == tn && strcmp(table[i][j], "-") != 0) {
83                 hasEps[eind] = states[i];
84                 eind++;
85             }
86             if (j == tn + 1) {
87                 if (strcmp(table[i][j], "y") == 0) {
88                     isfinal[i] = 1;
89                 } else
90                     isfinal[i] = 0;
91             }
92         }
93     }
94
95     for (int i = 0; i < eind; i++) {
96         eclsr[0] = '\0';
97         clsind = 0;
98         eClosure(hasEps[i]);
99         for (int j = 0; j < sn; j++) {
100             for (int k = 0; k < tn; k++) {

```

```

101     for (int l = 0; l < strlen(table[j][k]); l++) {
102         if (table[j][k][l] == hasEps[i]) {
103             for (int p = 0; p < clsind; p++) {
104                 int flag = 1;
105                 for (int o = 0; o < strlen(table[j][k]); o++) {
106                     if (table[j][k][o] == eclsr[p]) {
107                         flag = 0;
108                         break;
109                     }
110                 }
111                 if (flag) {
112                     table[j][k][strlen(table[j][k])] = eclsr[p];
113                 }
114             }
115             break;
116         }
117     }
118 }
119 }
120 int thisind = 0, fin = 0;
121 for (int k = 0; k < sn; k++) {
122     for (int l = 0; l < clsind; l++) {
123         for (int p = 0; p < sn; p++) {
124             if (states[p] == eclsr[l] && isfinal[p] == 1) {
125                 fin = 1;
126                 break;
127             }
128         }
129         if (fin == 1)
130             break;
131     }
132     if (states[k] == hasEps[i])
133         thisind = k;
134 }
135 for (int j = 0; j < clsind; j++) {
136     for (int k = 0; k < sn; k++) {
137         if (eclsr[j] == states[k]) {
138             for (int l = 0; l < tn; l++) {
139                 for (int o = 0; o < strlen(table[k][l]); o++) {
140                     int ffl = 0;
141                     for (int p = 0; p < strlen(table[thisind][l]); p++) {
142                         if (table[thisind][l][o] == table[k][l][p]) {
143                             ffl = 1;
144                             break;
145                         }
146                     }
147                     if (ffl == 0 && table[k][l][o] != '-') {
148                         if (table[thisind][l][0] == '-')
149                             table[thisind][l][0] = table[k][l][o];
150                         else

```

[illegible]

```

200         fff = 0;
201         break;
202     }
203 }
204 if (fff == 1 && table[flgg][j][p] != '-') {
205     curres[curlen] = table[flgg][j][p];
206     curlen++;
207     resfinal[resind] = isfinal[flgg];
208
209     }
210     }
211 }
212 }
213 if (curlen == 0)
214     continue;
215 flgg = 1;
216 strcpy(sstr, curres);
217 sortString();
218 strcpy(curres, sstr);
219 for (int k = 0; k < rescount; k++) {
220     curres[curlen] = '\0';
221     strcpy(sstr, resStates[k]);
222     sortString();
223     strcpy(resStates[k], sstr);
224     if (strcmp(resStates[k], curres) == 0) {
225         flgg = 0;
226     }
227 }
228 if (flgg == 1) {
229     strcpy(resStates[rescount], curres);
230     rescount++;
231 }
232 strcpy(restable[resind][j], curres);
233 }
234 resind++;
235 }
236 for (int i = 0; i < tn; i++) {
237     printf("\t%c", terms[i]);
238 }
239 printf("\tFinal?\n");
240 for (int i = 0; i < rescount; i++) {
241     if (i == 0)
242         printf("Start state ");
243     else if (i + 1 == rescount)
244         printf("Final State ");
245     else
246         printf(" ");
247     printf("%c\t", ('A' + i));
248     for (int j = 0; j < tn + 1; j++) {
249         if (j == tn) {

```

```
250         if (resfinal[i]) {
251             printf("YES\n");
252         } else
253             printf("N0\n");
254     } else if (strcmp(restable[i][j], "") == 0 || strcmp(restable[i]
[j], "-") == 0)
255         printf("NULL\t");
256     else {
257         for (int k = 0; k < rescount; k++) {
258             if (strcmp(resStates[k], restable[i][j]) == 0) {
259
260                 printf("%c\t", ('A' + k));
261                 break;
262             }
263         }
264     }
265 }
266 }
267 printf("\n");
268 }
```

| 11 LEX analyzer to find lexemes

| Algorithm

| Step 1: Define Tokens and Patterns

1. Token Definitions:

- `lite` : Matches 2-digit integers.
- `inv` : Matches invalid identifiers starting with a number.
- `ident` : Matches valid identifiers starting with a letter.
- `lite2` : Matches string literals within double quotes.
- `rel` : Matches relational operators (`<=` , `<` , `>=` , `>`).
- `key` : Matches keywords like `int` , `void` , `char` , `if` , `main` , `printf` .

2. Increment Line Number:

- Define a variable `no` initialized to 1 to track the line number.

| Step 2: Implement the Token Matching Rules

For each pattern defined above:

1. Integer (`lite`):

- Match integer tokens (two digits).
- Print the token as an integer type.

2. Literal (`lite2`):

- Match string literals.
- Extract content between quotes by removing the leading and trailing quotation marks.
- Print as a literal token.

3. Invalid Identifiers (`inv`):

- Match invalid identifiers that begin with a number.
- Print as "Invalid".

4. Relational Operators (`rel`):

- Match relational operators.
- Identify and print the specific relational operator (e.g., `LE` for `<=` , `LT` for `<` , etc.).

5. Arithmetic Operators (`+` , `-` , `*` , `/`):

- Match arithmetic operators.
- Print the specific operator type (e.g., `ADD` for `+` , `SUB` for `-`).

6. Assignment Operator (`=`):

- Print as an assignment operator (`EQ`).

7. **Keywords** (`key`):

- Match and print recognized keywords.

8. **Identifiers** (`ident`):

- Match valid identifiers that start with a letter.
- Print as an identifier.

| **Step 3: Update Line Number for Each New Line**

- Increment `no` by 1 whenever a newline (`\n`) is encountered.

| **Step 4: Ignore Unrecognized Characters**

- Discard any unmatched characters (defined by a general `.` rule).

| **Step 5: Main Function to Execute Lexer**

1. Open `input.txt` file for reading.
2. Print the table headers ("Lexeme", "Line", "Token").
3. Run `yylex()` to process the tokens in the file.

| **code**

11.1

```
1  %{
2      #include <stdio.h>
3      int no=1;
4  %}
5  lite [0-9][0-9]*
6  inv [0-9][a-zA-Z0-9]*
7  ident [a-zA-Z][a-zA-Z0-9]*
8  lite2 \"[a-zA-Z][a-zA-Z0-9]*\"
9  rel <=|<|>|=|>
10 key int|void|char|if|main|printf
11
12 %%
13
14 {lite} {printf("%s\t%i\tInteger\n",yytext,no);}
15 {lite2} { char aa[10],bb[10];
16           strcpy(aa,yytext);
17           for(int i=1;i<strlen(aa)-1;i++){
18               bb[i-1]=aa[i];
19           }
20           printf("%s\t%i\tLiteral\n",bb,no);
21
22       }
23 {inv} {printf("Invalid:\t%s\n",yytext);}
```

```

24  {rel} {
25      printf("%s\t%i\tRelational Operator, \t",yytext,no);
26      if(strcmp(yytext,"<=")==0){printf("LE");}
27      if(strcmp(yytext,"<")==0){printf("LT");}
28      if(strcmp(yytext,">=")==0){printf("LE");}
29      if(strcmp(yytext,">")==0){printf("LT");}
30      printf("\n");
31  }
32  "+"|"-"|"*"|"/" {printf("%s\t%i\tArithmetic opeator, ",yytext,no);
33      if(strcmp(yytext,"+")==0){printf("ADD");}
34      if(strcmp(yytext,"-")==0){printf("SUB");}
35      if(strcmp(yytext,"/")==0){printf("DIV");}
36      if(strcmp(yytext,"*")==0){printf("MUL");}
37      printf("\n");
38  }
39  "=" {printf("%s\t%i\tAssignment operator, EQ\n",yytext,no);}
40  {key} {printf("%s\t%i\tkeyword\n",yytext,no);}
41  {ident} { printf("%s\t%i\tIdentifier\n",yytext,no);}
42  "\n" {no++;}
43  . {}
44
45  %%
46
47  int main()
48  {
49      yyin=fopen("input.txt","r");
50      printf("Lexeme\tLine\tToken\n");
51      yylex();
52  }

```

| 12 Simple calculation using YACC

| algorithm

This algorithm uses a **YACC (Yet Another Compiler Compiler)** parser to evaluate arithmetic expressions with addition, subtraction, multiplication, division, and modulus operations. The **LEX (Lexical Analyzer)** is used to identify and tokenize numeric values from the input.

| Steps:

1. Define Tokens in YACC and LEX

- Define tokens (such as `NUMBER`) in both YACC (`12.y`) and LEX (`12.l`) files.
- Each token corresponds to components of the arithmetic expression (e.g., numbers and operators).

2. Create LEX Rules (`12.l` file)

- Write patterns for recognizing numeric values:
 - `[0-9]+` : Matches integers. The value is converted using `atoi()` and stored in `yyval`.
- Skip tabs (`[\t]`) and handle newline characters to end the input.
- For any other character, return it directly (used for operators like `+`, `-`, `*`, etc.).

3. Create YACC Grammar Rules (`12.y` file)

- Define precedence and associativity for operators:
 - `+` and `-` (left-associative).
 - `*`, `/`, and `%` (left-associative).
- Write grammar rules for arithmetic expressions:
 - Use a recursive grammar to handle expressions like `E '+' E`, `E '-' E`, etc., with actions to compute results.
- Include an `ArithmeticExpression` rule as the start symbol, which triggers evaluation and printing of the final result.

4. Define Semantic Actions for Each Rule

- Actions are embedded within `{ }` after each rule. They determine how results are calculated:
 - `$$` holds the result of the rule.
 - `$1`, `$2`, etc., represent matched subcomponents of the expression (like numbers or intermediate results).

5. Error Handling

- If an error is detected in parsing, the `yyerror` function is called, setting `flag` to 1.
- After parsing, check `flag`. If `flag` remains 0, the expression is valid and processed; otherwise, it's invalid.

6. Compile and Run

- Compile the YACC file with `yacc -d 12.y`.
- Compile the LEX file with `lex Pg12.l`.
- Use `gcc` to combine the generated `lex.yy.c` and `y.tab.c` files.
- Execute the output file `./a.out` to start the evaluator.

7. Enter Input and Receive Output

- The program prompts for input.
- Enter an arithmetic expression (e.g., `3 + 4 * (5 - 2)`).
- The program outputs the evaluated result or displays an error if the expression is invalid.

code

12.y

```

1  %{
2      #include<stdio.h>
3      int flag=0;
4  %}
5  %token NUMBER
6  %left '+' '-'
7  %left '*' '/' '%'
8  %left '(' ')'
9
10 %%
11
12 ArithmeticExpression:
13     E { printf("output:%d\n", $$); return 0; };
14 E:
15     E '+' E {$$=$1+$3;}
16     | E '-' E {$$=$1-$3;}
17     | E '*' E {$$=$1*$3;}
18     | E '/' E {$$=$1/$3;}
19     | E '%' E {$$=$1%$3;}
20     | '(' E ')' {$$=$2;}
21     | NUMBER {$$=$1;}
22     ;
23
24 %%
25 void main() {
26     printf("\nEnter the input:\n");
27     yyparse();

```

```
28         if(flag==0)
29             printf("Expression is Valid\n");
30     }
31     void yyerror() {
32         printf("Expression is invalid\n");
33         flag=1;
34     }
```

12.1

```
1  %{
2      #include<stdio.h>
3      #include "y.tab.h"
4      extern int yylval;
5  %}
6
7  %%
8
9  [0-9]+ { yylval=atoi(yytext); return NUMBER;}
10 [\t]   ;
11 [\n]   return 0;
12 .      return yytext[0];
13
14 %%
15
16 int yywrap() {
17     return 1;
18 }
```